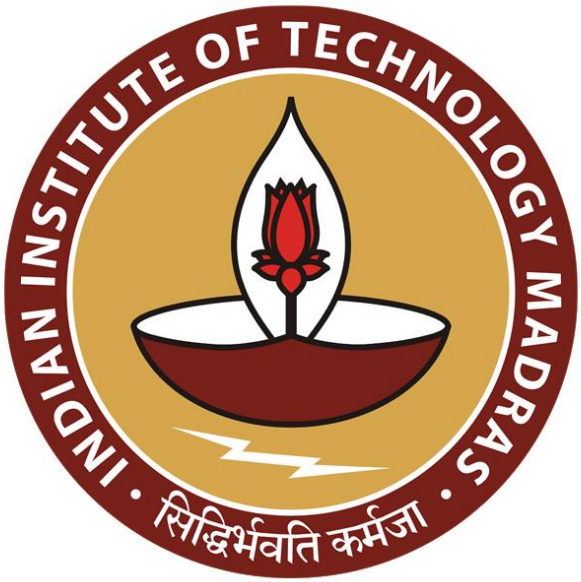


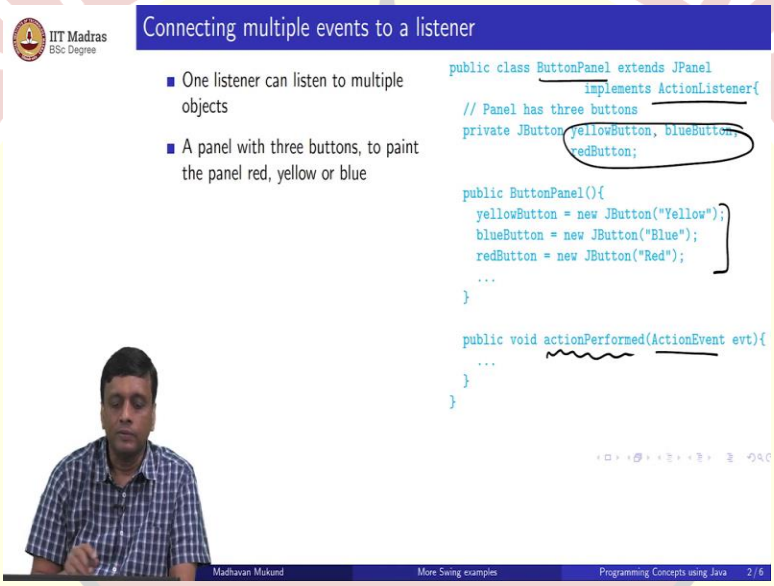
IIT Madras

ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical institute
Indian Institute of Technology, Madras
More Swing Examples

So, we saw a simple button in Java. So, now let us look at more slightly more elaborate examples of how things work in Swing.

(Refer Slide Time: 00:23)

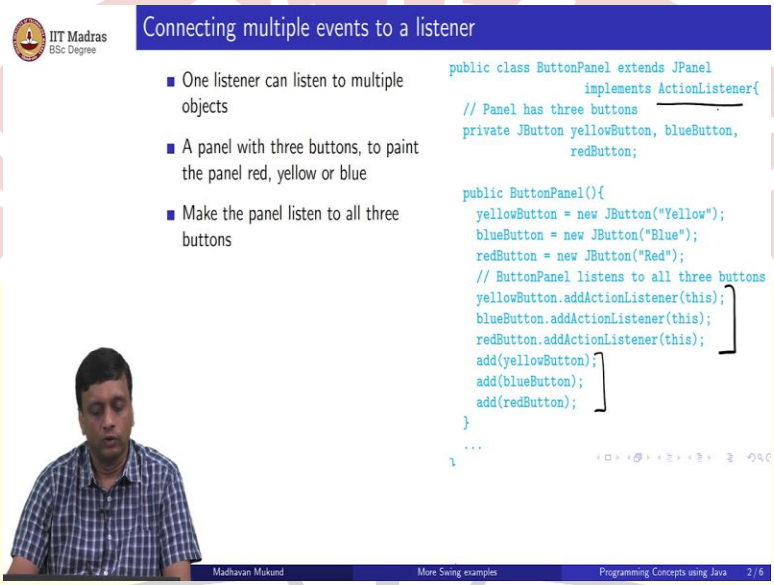


So, remember that we said that swing is very flexible about connecting events to listeners. So, in the button example, we had one button, which did one thing, and it was listened to by one panel, because it was just a very simple example. And that one panel did one thing that we told it to, which was to paint the background red when the red button was pressed.

But one listener can now listen to multiple objects. So, for example, supposing instead of that one button, we had three buttons in that panel, it had a button labeled red button labeled blue, and a button labeled yellow. So, we have three buttons now in the panel. And of course, now the interpretation of this is that the label indicates what is to be done. So, each button generates an action performed event. And that has to be handled, I mean, an action event and that so generates an action event. And that has to be handled by an appropriate listener, which is doing action performed. But we want to all do this all in one panel.

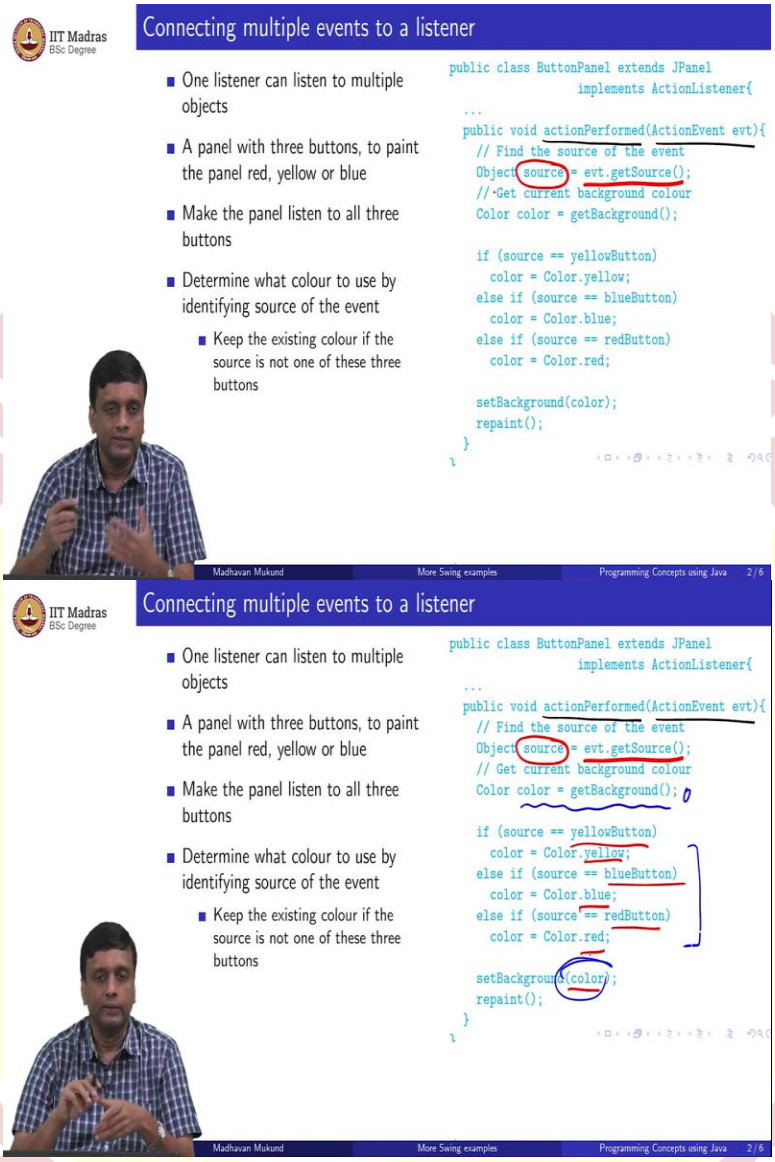
So, this panel should now listen to all these three buttons, and do the appropriate thing. So, we have this one panel as before, which is a JPanel, which implements action listener. But now it has three types of buttons. And now it must disambiguate, when it comes to action perform which button was clicked.

(Refer Slide Time: 01:42)



So, I make the panel listen to all three buttons? So, I add to each of the buttons, I add the action listener to be this current panel, the embedding plan, and then I add all the three buttons to the panel. So, up to this point, there is no problem. But the problem we have to now do is to decide how to implement this action listener part. So, that if I get a button, press event, from the yellow button, I must turn it yellow from the blue button, I must turn it blue and so on.

(Refer Slide Time: 02:10)



So, this is where we said that the event actually is an object, it has structure and it has information in it so we can analyze the structure. So, one of the things that we can analyze is, which button was the source of the event? So, we can get from an event. So, this is the action performed now.

So, it gets this action event. And what it does is it queries it says, what was the source of this event? So, the source of the event is some object. And now, remember that this is all sitting inside the same button panel. So, the same button panel now has these three buttons. So, these are all local instance variables of this button.

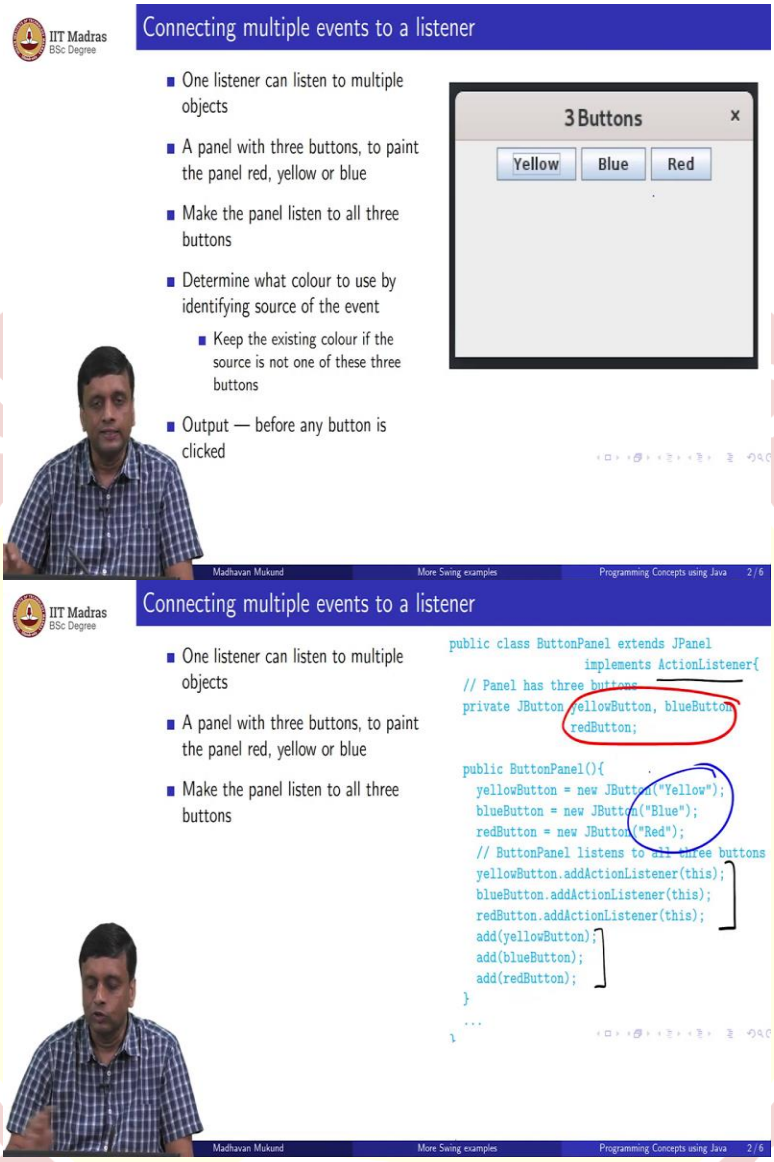
So, these are known to it. So, this button panel will now ask itself is the source the yellow button, or is the source the blue button, or is the source the red button, so the source is the yellow button, it will turn the color to be yellow. If it is the blue button, it will turn the color to be blue, if it is red button, return the color to be red.

And then it will set the background color. What if it is none of these three, what if there was a button maybe it is getting some somewhere else it is also listening to other things, then that should not affect the color. So, what we do before this is we get the existing color. So, that this color variable has the current color as a background.

And then if these three ifs all fall through without triggering, then we will just go back and restore the old color. So, the default is to restore the old color unless exactly one of these three events. Now, the reason is that because now we have shown in this case that a listener can listen to more than one button. So, here we are listening to these three buttons, maybe this listener is also listening to buttons elsewhere. And those buttons should not change the color of this panel. They are some other effect. So, therefore, we have to have this background thing of keeping the color the same, in case there is no change.

And finally, we repeat. So, this is just an extension of the single button case and the single button case, we did not have to analyze where the event came from. Because it came from only one button, we are only listening to one button if that one button is pressed, we painted the background rate. Here there are three buttons. So, when we get an event, we have to first check which of these three buttons it was by checking its source. And depending on what the source corresponds to, we will choose the appropriate color and then repeat.

(Refer Slide Time: 04:35)

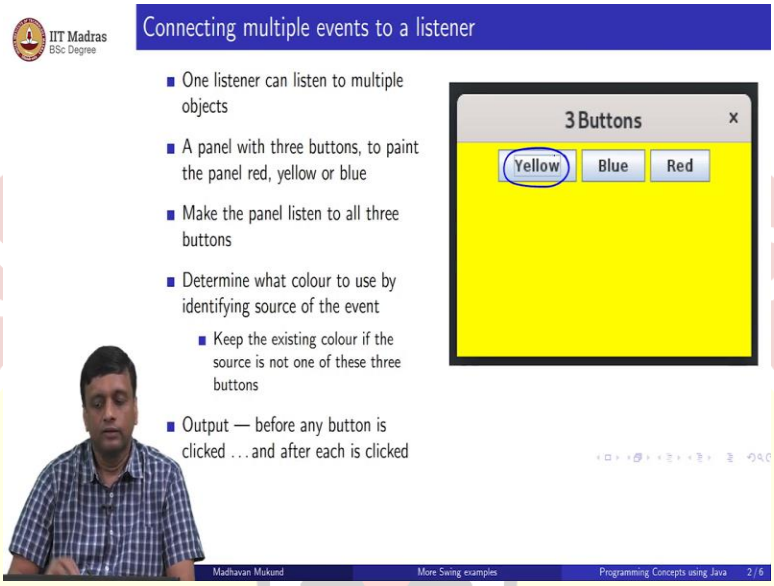


So, this is what this thing looks like of course then you have to take this panel and put it in a frame and so on which is exactly the same as with a single button case. So, I have not shown you the code for that. But there is essentially no change except you have to obviously create an object of this type rather than the other type.

So, now when we start up this thing, we have a kind of by default, swing puts this gray color panel it looks like and it shows you these three buttons, remember the buttons had these names red, yellow, blue, and red. So, it puts these three buttons, and it kind of automatically decides how to position them. So, it puts them side by side depends on the size of the frame and the size of the

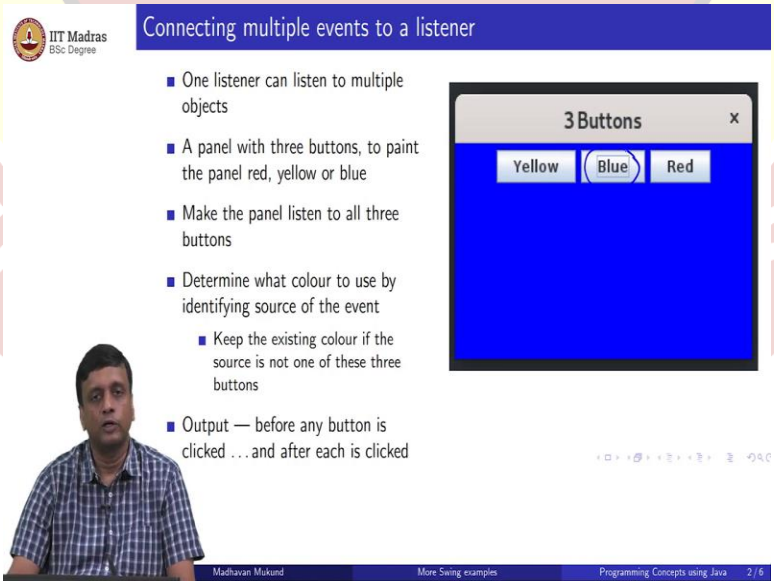
thing that you are making visible, how it will put them, if you make a frame very narrow, it might put them one on top of the other, and so on, now when we press the yellow button.

(Refer Slide Time: 05:21)



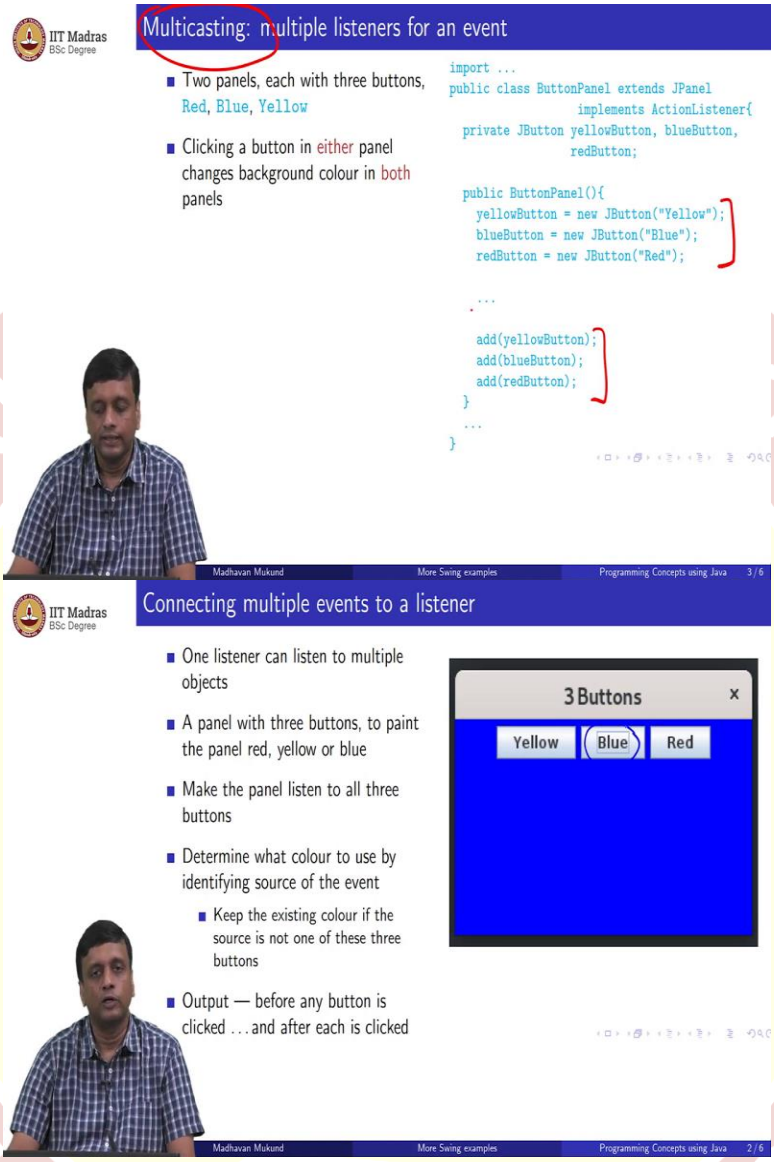
We get a yellow thing. And you can also see that, the yellow button has indication that it was pressed and now when we press the blue button.

(Refer Slide Time: 05:32)



Then the background turns blue, because that is what the listener says. And notice that we have an indication on the screen that the blue button was pressed and same for the red button.

(Refer Slide Time: 05:42)

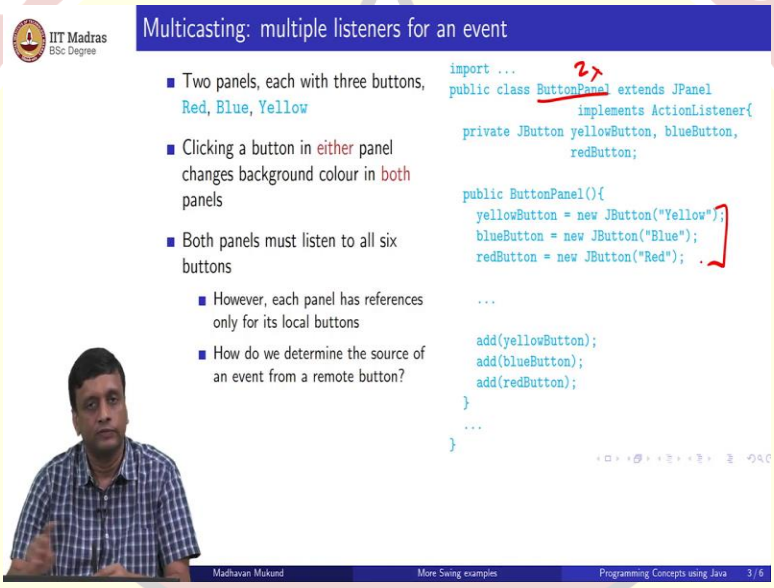


So, it should also be red, but something went wrong. So, the same thing will work for the red button as well. So, now this is an example where we had one panel, listening to three buttons, so we had multiple generators of events and a single listener. Now, what if we have two panels and each panel has three buttons of the same type red, blue, yellow. But now the twist is that if I press the red button in one panel, it must change both panels to red. If I press the yellow button in any panel, it must change both panels to yellow.

So, the event now must notify not one object or two objects. Earlier, we had one object, which was listening to multiple buttons. Here we have one button, which must tell multiple panels. So, this is what is called multi casting? So, multi casting says that we have multiple listeners for an event.

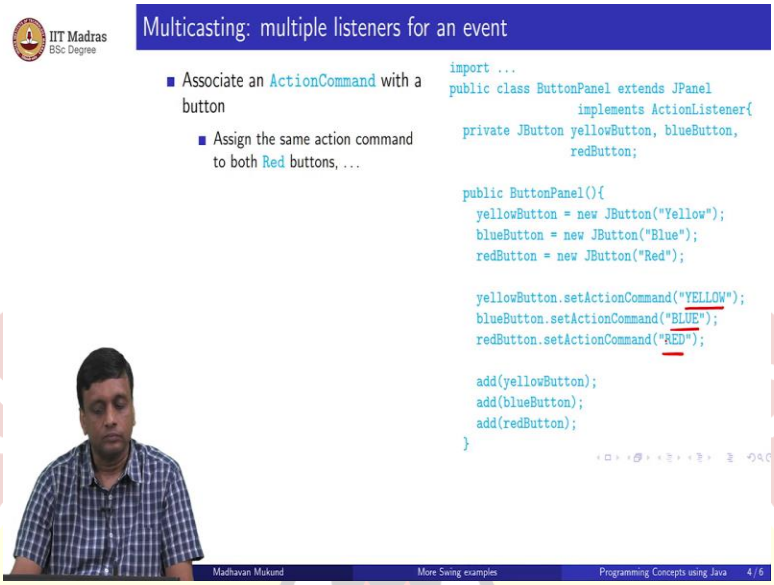
So, this is an interesting idea that we do not have to just send the signal in one direction, we can send it to more than one. So, here is our thing again. So, we have these three buttons, and we are going to add them. But between that we have to do something about setting up their identity so that we can do this.

(Refer Slide Time: 07:08)



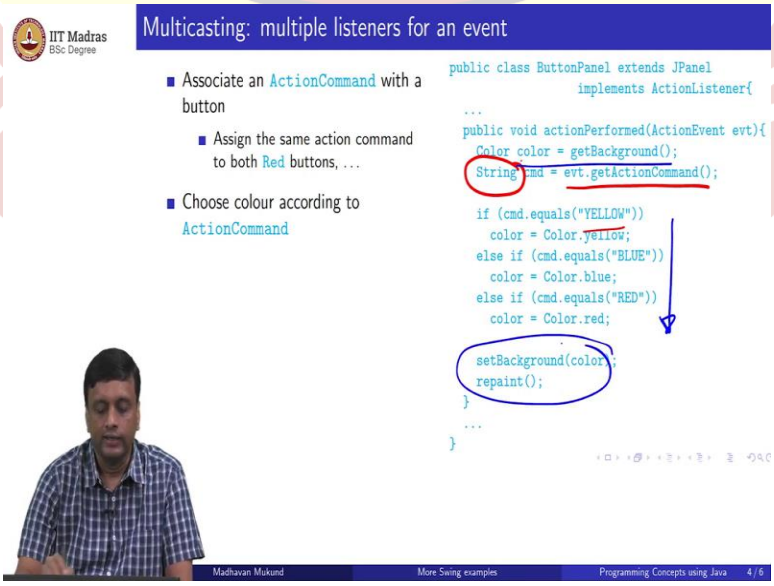
So, now both panels must listen to all six buttons. So, there are three buttons in one panel, there are three buttons on another panel and all buttons must be listened to by both panels. But each panel only knows about its own buttons. So, we cannot use this get source to refer to buttons, which are from the other panels because we do not know their identity, the objects in the other panel are not known to this panel. So, how do we determine the source of an event from a button which is on the other panel? So, remember, there are two such things and each has one copy of the buttons. So, you do not have the identity, they are private to the other button other panels, you do not have the copies of the other things availability.

(Refer Slide Time: 07:49)



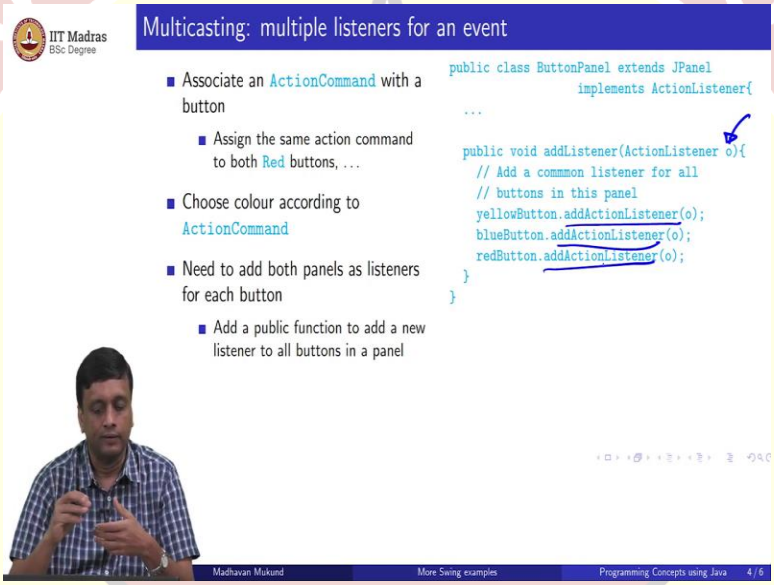
So, what we can do is we can associate a label to the event earlier we were looking at the source of the event. So, now in some sense, we are attaching a type of our description, we are attaching a description to the event, which is called an action command. So, we are saying pressing this button conveys the string. So, the strings that we have correspond to the colors that we want. So, capital, yellow capital, it could be anything. But it is something now where now when we get the event, we can ask what is the action associated with this event. And it will tell us the action is either the string yellow, or the string red or the string.

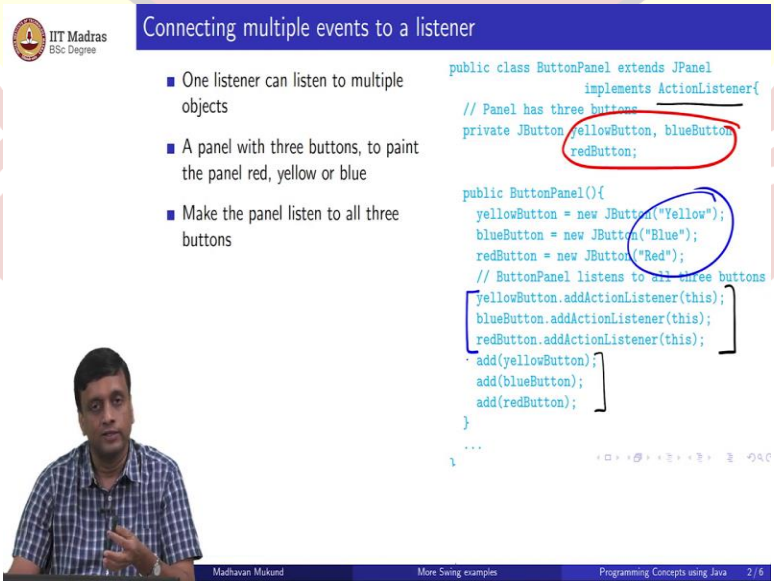
(Refer Slide Time: 08:27)

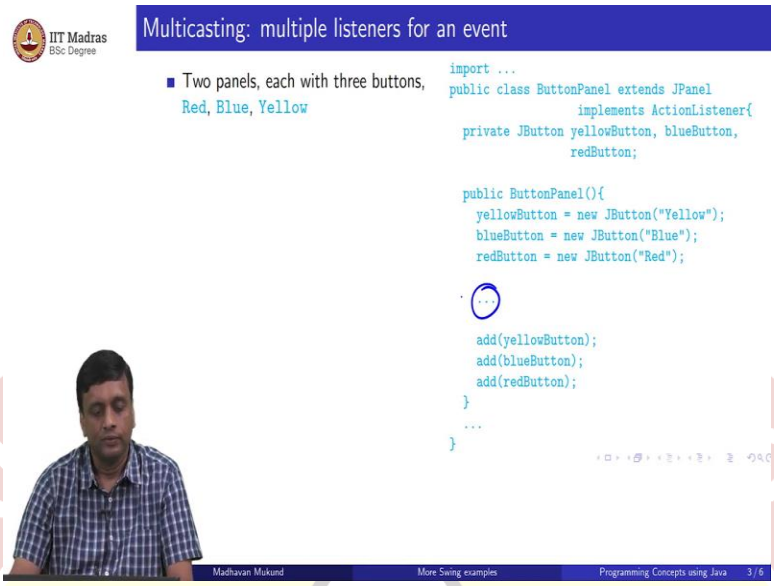


So, now if I look at the other side, if I look at the action performed thing, instead of asking for the source, as we did before, now we will say from the event, which I just received, give me the action command, and this is a string. So, I store it as a string locally. And then I compare the string, if the string is yellow then set the color to yellow, and so on. And as before, we start with the background color here, so that if it is not one of these three things, I do not change the color. Otherwise, I change the color according to the string that I get from the action command. And then I set the background color and I repeat.

(Refer Slide Time: 09:04)







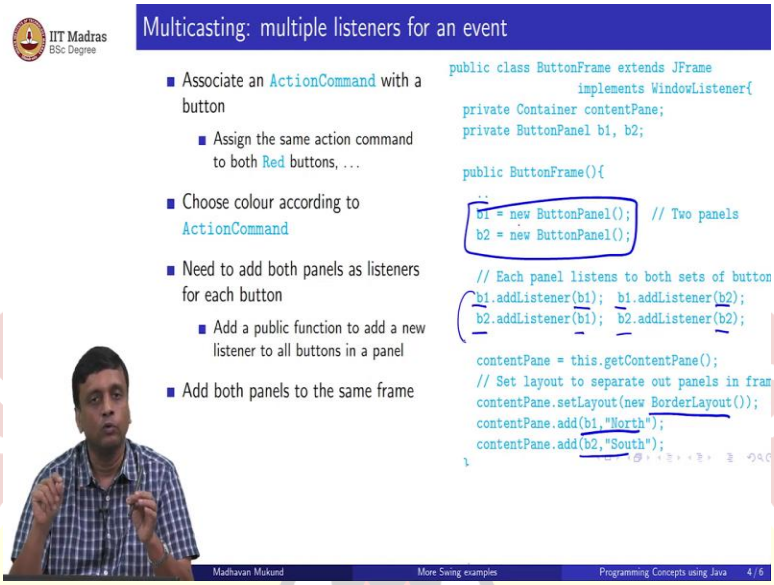
So, there is one more thing which is I have to make sure that each button notifies both the panels. So, earlier, if you remember, we had done that right at the time when we set up the button. So, if you go back here, when we created the button before we created the button, we just made the enclosing panel a listener.

But that is not enough because I also need to make the other panel which is not known to this panel at this point a listener so that is why we did not put this here, this is a place where we would have associated listeners, but we did not put it here. Because at this point, we cannot just associate this panel we have to associate both panels**.**

So, the solution to that is that we actually create a separate public function inside this panel class which allows it to receive the name of a button to which it must listen. So, we can add a listener. So, earlier, we had a button to which we added a listener. Now, we are adding to the listener, we are adding a button.

So, what we say is that if I get a button Sorry, I am adding, adding a panel to a button. So, I give you a panel. And what it does is it adds that panel to all the three buttons. So, we have this addActionlistener, which takes one button and associates a listener. Now we want to associate this with panel one and panel two. So, I will call this function once with this panel and once with another panel.

(Refer Slide Time: 10:42)



So, if I look at now how I create this inside the frame, now the frame changes a little bit, because in the earlier case, I just had to create a panel and be done with it because the panel setup the listener itself. But now the frame has to do this extra work of connecting each panel to the other. So, I create these two button panels.

So, this creates three buttons on one panel, three buttons and other panel. Now, I have to say tell these two panels that each button in your panel has to listen to the other thing also. So, I have to take b1 and add itself, as a listener. So, the b1 buttons all talk to b1 panel, I have to take b1 and add b2 as a listener.

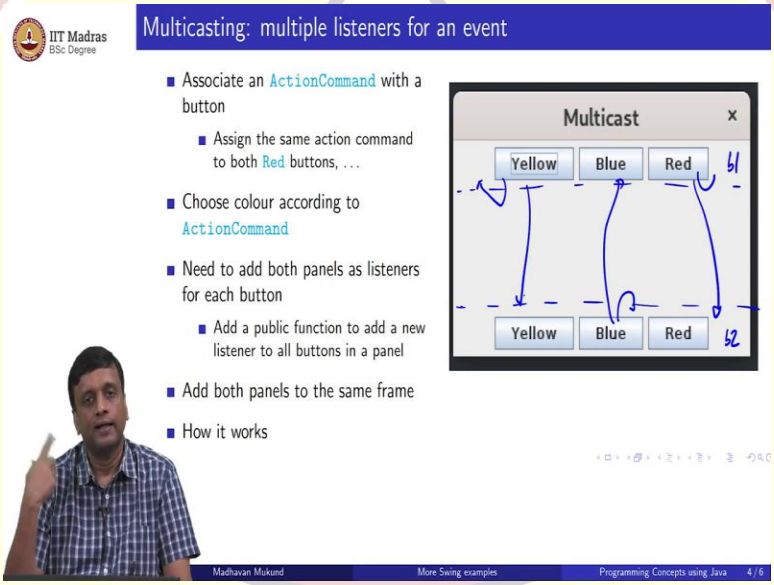
So, the b1 panel also notifies all the b1 buttons also noticed by all the panel b2. And similarly, b2 has to notify b1 and b2 has to notify b1. So, each button now the frame has the job of creating these two panels and telling these two panels to talk to each other. That is every button and you are thing must talk to the other guy and yourself and vice versa.

So, this is what sets up the multicasting. So, the multicasting is not set up in the panel itself, but by the frame that creates the panels. Now, there is another small thing we talked about the layout, we said when you put three buttons really depends on how these things are positioned with respect to the size of the frame. So, you can add some extra information to the frame. So, for instance, you can tell the frame that you want to layout in a particular way so that you can position things in the frame.

So, this has something you can look it up in the Java documentation, so what kind of layouts are there, but the simplest layout allows you to put something at the top bottom east and west source left and right. So, it is called north south east west. So, it says the first panel I am going to put in the North. The second panel I am going to put to the south.

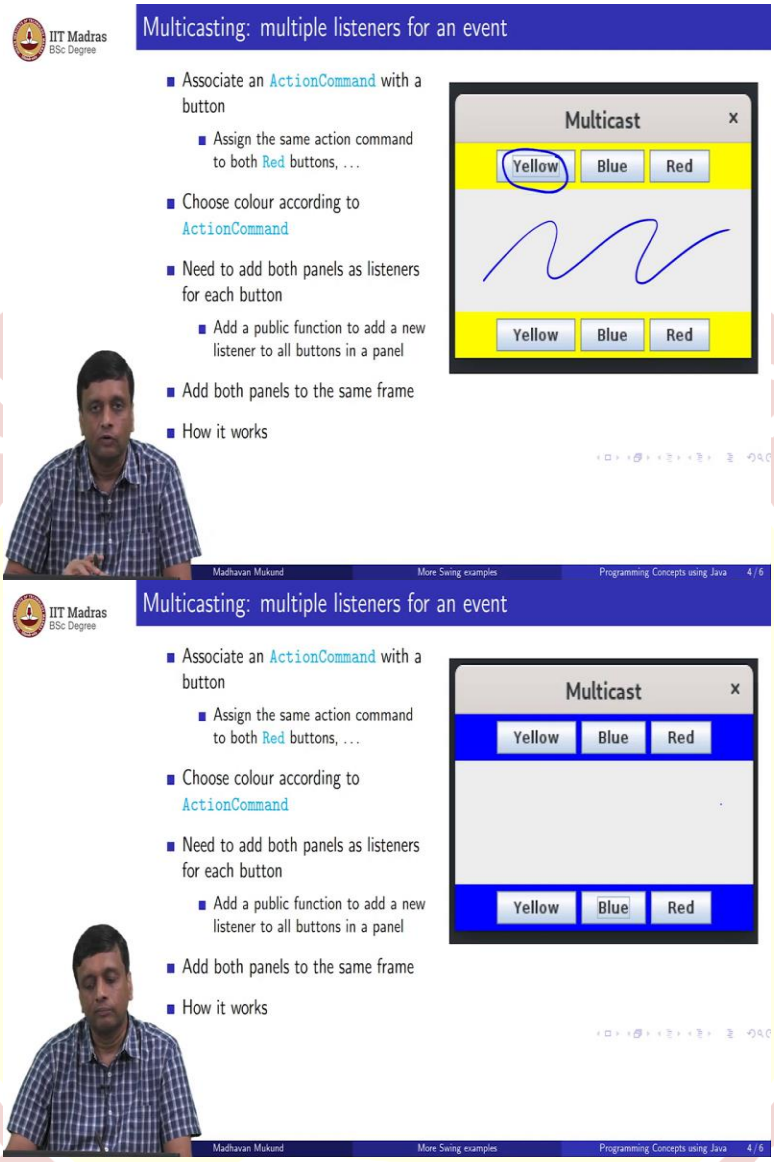
So, at this point, now I have created two panels, I have made them talk to each other. And I position one and the top one and the bottom. And each of them when it receives an event, it will look at the type of the event as given by the action command, whether it came from its own button or from the other side it look at the string, it gets yellow, red or blue and do the appropriate.

(Refer Slide Time: 12:39)



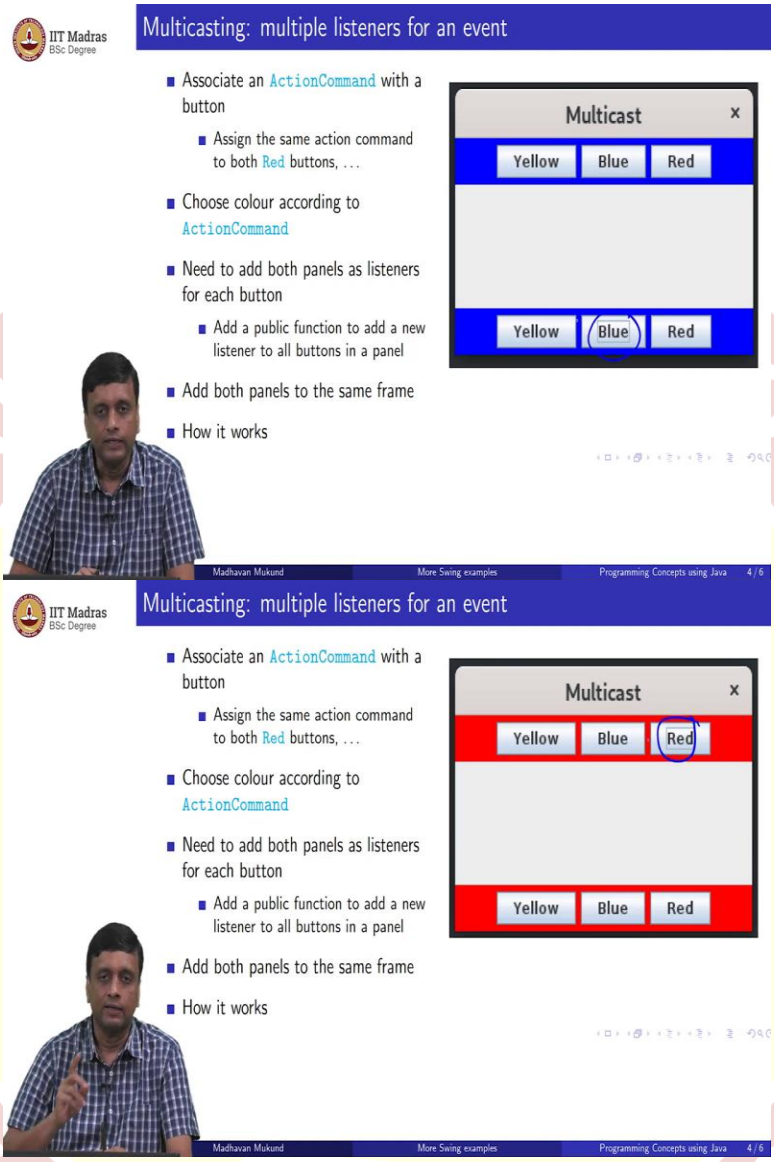
So, this is how it works. So, first of all, you cannot see the panels right now. But there is an invisible line here and here, which corresponds to the boundaries of the two panels, because I have pushed one to the north one to the south. So, there are 6 buttons, three belonging to the panel one. So, this is my b1. And three belong to a panel two and each of these talks. So, this read talks to here and talks to here this, blue talks to here and talks to this yellow talk to itself and talks to him. So, each of these buttons notifies the panel which is sitting and the panel on the other side. So, if for instance, I press the yellow button.

(Refer Slide Time: 13:14)



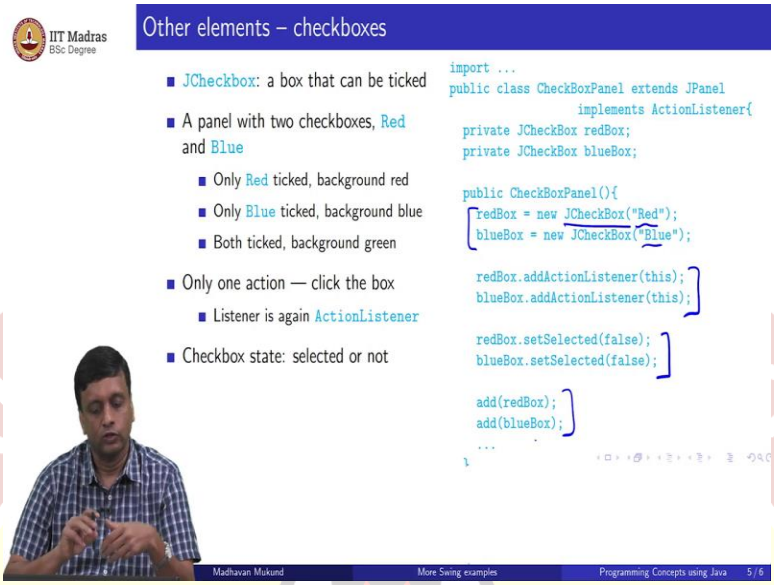
Then both these panels become yellow. And now I can see the boundary because only the panel part is yellow. So, this is the part in between which does not belong to either panel. Similarly, if I press the blue. Then both of them become blue. And if I press so now, I also want to emphasize that here the yellow on the top was pressed, you can see the square around the yellow.

(Refer Slide Time: 13:33)



Here, the blue on the bottom has been pressed. And you can see the gray rectangle around the blue. And now I press the red on the top. And again, it works. So, I press either one. And according to the label on the button that I see, it generates internally a string of the type that the panel understands. And that panel will decide based on that string, which color to change the background. So, this is multicasting, so you can take one event generating object and spread the event effect to multiple listeners.

(Refer Slide Time: 14:06)



So, finally, we have been working only with buttons. So, just to illustrate, that this is a kind of generic toolkit. So, swing has objects of various different kinds that you would want to put into a typical graphical user interface. Let us look at something called a checkbox? A checkbox is just a box that can be ticked, if you have ticked any online form. You have seen this you can ticked one or both, or any so it is not like a button where if you selected the other becomes unselected.

So, let us look at something which has now two checkboxes one called red and blue. And now the thing that we will ask is that if the red is ticked, then make the panel red. If the blue is ticked then make the panel blue but because it is not exclusive, it could be that both red and blue are ticked. In which case, we will make it green to indicate that both have been ticked. So, this is the desired behavior. We have, two checkboxes, and if both are ticked, then we will make it green. Otherwise, we will change the color according to the one that is ticked.

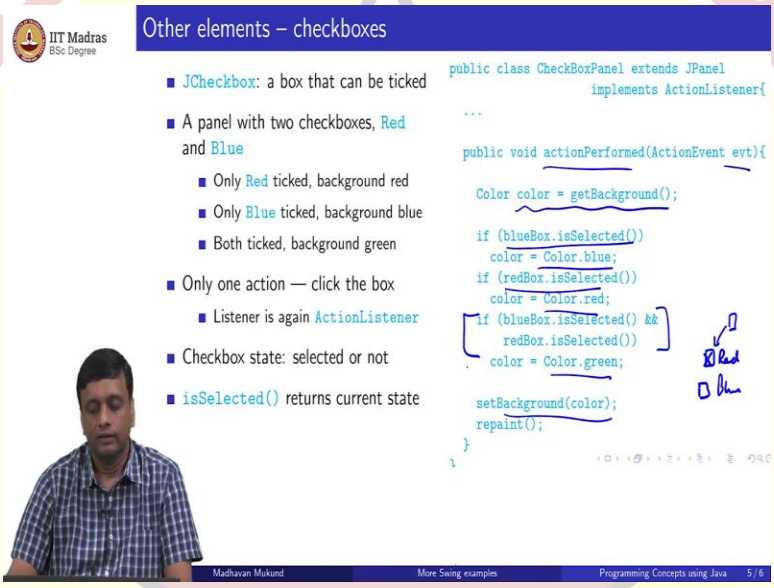
So, first of all, we have to have a way of indicating whether a checkbox is ticked or not. So, a checkbox first of all, when it is ticked, it generates an action, an event, which is an action listener, like a button, but the checkbox itself. Now, unlike a button, which every time I click it, it gets clicked, and then it remains in the same state. So, I can click it again, a checkbox is not like that, if I tick it, then it becomes ticked.

And if I tick it again becomes un ticked. So, I have to remember whether the checkbox is ticked or not, which is normally called selected. So, with a checkbox is selected or not. So, when I create

this, I set up these two boxes, which are, like we said, like J-button, J-panel J-frame, J-checkbox, I set up these two buttons with the appropriate label, I make them both talk to the enclosing panel, I initialize them both to be unselected and then add them to the panel.

So, that is the first part is simple enough, it is like adding two buttons, except I am adding two checkboxes. And in unlike two buttons where I do not have to say anything more, when I add a checkbox, I have to decide in the initial state with a checkbox is selected, that is ticked or it is not ticked.

(Refer Slide Time: 16:15)

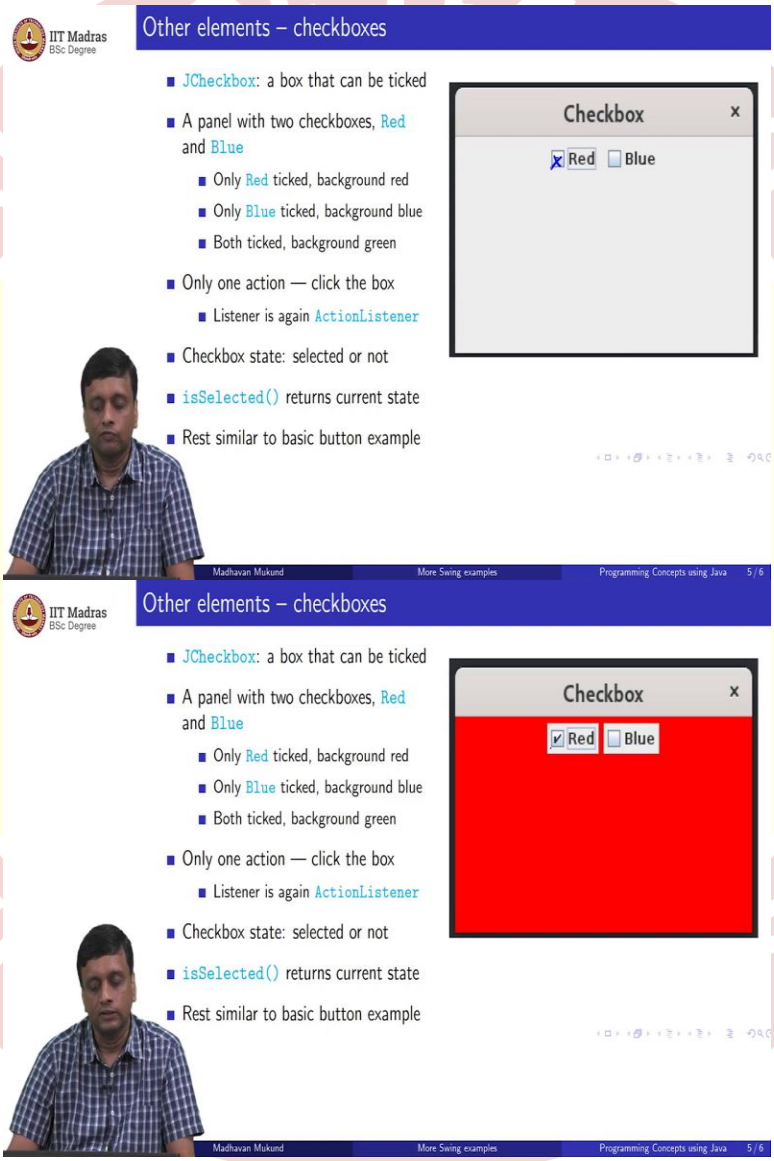


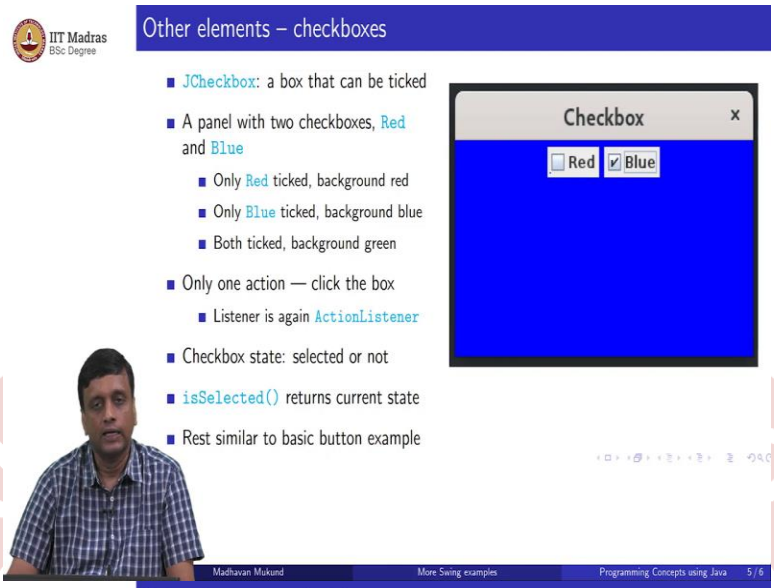
So, now in the listener side, the way it will get when it is notified, it has to see what was selected. So, the action performed, it does not have to do anything really with the event. It will rather look at the state of the checkboxes is in this case. So, it will say if the blue box is selected, then set the color to blue. If the red box is selected, set the color to red.

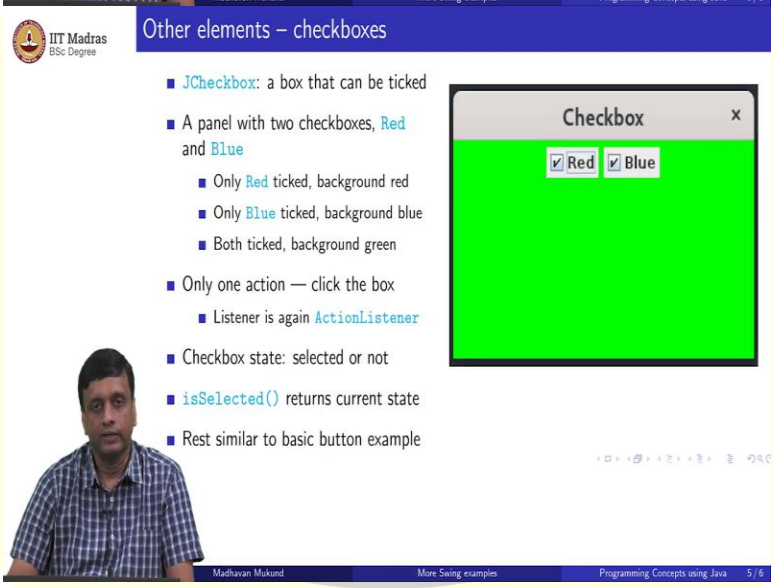
So, if either of these selected, only one of them is selected, then if it is blue, it will set to blue, if it is red to set to red, and then it will say if both are selected, then set the color to green. And as usual, we start with the existing background color and then we replace it. So, in particular, what this means is that if I had say the red checkbox and the blue checkbox and then I did this, so then I made it red.

Then if I uncheck this and make it again empty, it will remain red why because it now red is not selected blue is not selected. So, it was red, so it will remain red. Similarly, if I make the blue, if you make it blue by ticking blue and then I un ticked the blue it will remain blue and if I ticked both because of this and it will make it green.

(Refer Slide Time: 17:29)



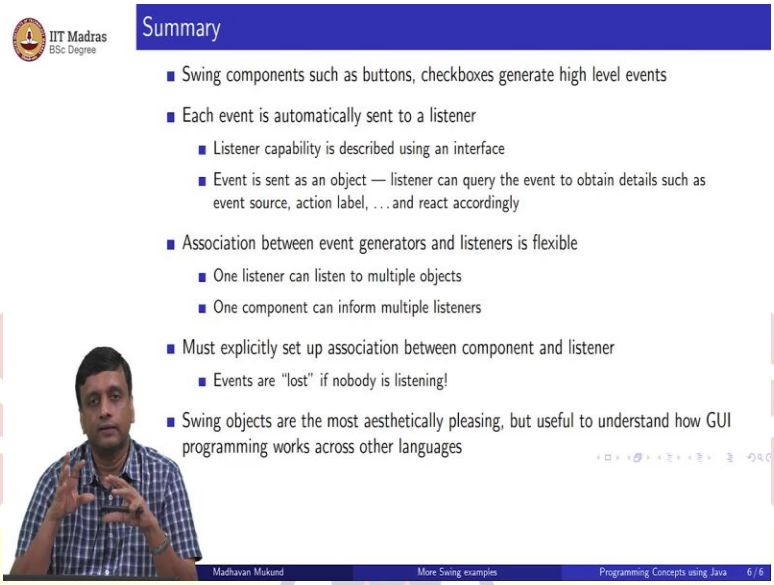




So, this is how it looks. So, we have a checkbox, the frame sets the label on the top and it has these two boxes in which I have this place which I can go and select which I can go and make this an x. So, if I make the red and x if I take the red box, then I will get a red frame, if I take only the blue box, I will get a blue frame and if I take both boxes, I will get a green frame.

So, this is just to illustrate that you can easily program things other than buttons. So, check boxes, you can make lists, you can make radio buttons, you can do various things, just everything that you normally see in a graphical form or in a graphical interface is actually available in swing for you to program.

(Refer Slide Time: 18:11)



So, the summary is that swing has this built in notion of components and these components like buttons and checkboxes and so on generate these high-level events depending on what type of object they are. And each event is automatically sent to a listener that is listening. And what the listener is supposed to do is describe through an interface depending on the nature of the component that you have.

And the listener can now take this event that it gets when the event happens, it is an object so it can look at this event and analyze it can say where did it come from? What is the action label associated with it and so on and react accordingly. So, this association is flexible as we have seen you could have one is too many, many is too one. So, you can multicasting all these things, but you have to do it explicitly.

So, if you do not connect a component to a listener, then the components events will get generated and just will just disappear. So, nobody is listening to them they will get lost. So, this is what we said you will get an unresponsive button. For example, nothing happens because there is nobody listening to it. So, you have to be it is the programmer's responsibility to do this. So, this is not true in all other programming languages in some programming languages, there is an implicit connection that is the something that creates a button will also listen to it. So, you will always be guaranteed that there is a listener but then you lose some flexibility.

So, this flexibility actually gives you the most general setting and then when you go to other languages you have to look at which level of flexibility that language supports. So, it is typically not the case that people program graphical interfaces in swing because swing is not the as we have seen it is a little cumbersome to do.

And it is also aesthetically not the most pleasing design so you do not have much choice about layout and colors and shades and shapes and so on. So, if you really want a cool looking design, which appeals to a graphic designer swing is not probably the best thing to do. So, we are not discussing swing as a graphical design tool, to layout beautiful web pages.

But to illustrate that, it has all the principles that you need to understand how these things work. What happens when you set up a graphical page, you have these components, you have these listeners, components, react, and generate events, and listeners react to them? And once you understand this, then you can pretty much understand the setup in any programming language or any kind of platform which allows you to design these things because they will all be there in some form, usually less flexible than swing, but possibly more aesthetically controllable.